AgentRoute: Kullanım Kılavuzu

Bu kılavuz projenin kurulumunu, her parçanın nasıl çalıştırılacağını ve çıktıların nerede olduğunu anlatır. Komutlar proje klasöründe (`mobile/`) çalıştırılır.

1. Gereksinimler

Ne	Neden	Not
Python 3.11 veya üstü	Simülatör, eğitim, deneyler, grafikler	3.12 ile test edildi
~2 GB disk	PyTorch + MiniLM modeli	
İnternet (ilk çalıştırma)	MiniLM modeli Hugging Face'ten iner (~90 MB)	Sonra cache'ten okunur
Node.js LTS (isteğe bağlı)	Sadece React log oynatıcısı için	nodejs.org'dan `.pkg` kurulumu

GPU gerekmez. Her şey CPU'da çalışır (MacBook Air M2 üzerinde denendi).

2. Kurulum

`uv` varsa (önerilen, hızlı):

```
uv venv --python 3.12 .venv
```

```
uv pip install --python .venv/bin/python -r requirements.txt
```

`uv` yoksa standart yol:

```
python3 -m venv .venv
```

```
.venv/bin/pip install -r requirements.txt
```

Ortamı etkinleştir (bundan sonraki komutlar `python` ile çalışır):

```
source .venv/bin/activate
```

Kurulumu doğrula. Testlerin hepsi geçmeli:

```
python -m pytest -q
```

3. Tek komutla her şey

```
python experiments/run_all.py --quick
```

`--quick` kısa bir duman testidir (birkaç dakika). Sırasıyla şunları yapar: τ ayarı, (model yoksa) kısa eğitim, bütün deneylerin küçük sürümü, grafikler, demo koşuları, animasyonlar, web oynatıcıya veri aktarımı.

Raporda kullanılan tam sürüm:

```
python experiments/run_all.py
```

Model zaten eğitildiyse eğitimi atlamak için `--skip-train` ekle. `--workers N` paralel süreç sayısıdır (varsayılan 6).

4. Adım adım

4.1 Kabul eşiği τ

```
python experiments/tune_tau.py
```

Train parafrazları üzerinde F1'i en yüksek yapan τ'yu seçer ve `results/tau.json` dosyasına yazar. Simülatör τ'yu buradan okur. Aynı dosyada test kümesi için top-1 doğruluk ve yanlış kabul oranı da var.

4.2 DQN ve GNN eğitimi

```
python experiments/run_train.py
```

Seçenek	Anlamı
`--kind mlp` / `--kind gnn` / `--kind both`	Hangi model (varsayılan: ikisi de)
`--episodes N`	Episode sayısı (varsayılan 300, `configs/default.yaml` → `dqn.episodes`)

Çıktılar:

- `results/models/dqn.pt`, `results/models/gnn.pt`: en iyi doğrulama getirisine sahip ağırlıklar
- `results/raw/train_mlp.csv`, `results/raw/train_gnn.csv`: öğrenme eğrisi verisi

Süre: MLP ≈ 20 dk, GNN ≈ 30 dk (M2). İkisini ayrı terminallerde paralel çalıştırmak süreyi kısaltır.

4.3 Deneyler

Her script bir parametreyi tarar, her (değer, yöntem, seed) için bir simülasyon koşar ve `results/raw/<ad>.csv` dosyasına yazar. Hepsi `--quick`, `--workers N`, `--config dosya.yaml` alır.

Script	Taranan parametre	Çıktı
`experiments/run_scale.py`	agent sayısı 100 → 10.000	`scale.csv`
`experiments/run_mobility.py`	mobil agent hızı 0–30 m/s	`mobility.csv`
`experiments/run_failure.py`	çöken router oranı 0–%30	`failure.csv`
`experiments/run_radius.py`	duyuru yarıçapı R = 1–6	`radius.csv`
`experiments/run_load.py`	yük çarpanı 0.5–4	`load.csv`
`experiments/run_ablation.py`	S-FIB özeti (mean / k / exact) ve intent header (float32 / int8 / binary)	`ablation_summary.csv`, `ablation_header.csv`
`experiments/run_handover_cost.py`	handover başına S-FIB mesajı vs tam duyuru turu (1.000 / 3.000 / 10.000 agent)	`handover_cost.json`

Yöntem adları: `agentroute` (DQN), `agentroute_gnn`, `greedy`, `flooding`, `registry`, `chord`.

4.4 Grafikler

```
python viz/plots.py
```

`results/figures/` klasörüne her grafiğin PNG ve PDF sürümünü yazar:

Dosya	İçerik
`similarity`	Parafraz isteklerin yeteneklere cosine benzerliği, τ çizgisi
`scale_success`	Agent sayısı vs başarı oranı
`scale_overhead`	Agent sayısı vs istek başına mesaj (log ölçek)
`scale_hops_time`	Agent sayısı vs hop sayısı ve tamamlanma süresi
`mobility_success`	Hız vs başarı
`failure_success`	Çöken router oranı vs başarı
`radius`	R vs başarı ve kontrol mesajı
`load`	Yük vs başarı ve süre
`learning_curve`	DQN/GNN eğitim getirisi ve doğrulama başarısı
`ablation_summary`	S-FIB özet türünün etkisi
`ablation_header`	Header sıkıştırmanın etkisi
`drop_reasons`	İsteklerin neden başarısız olduğu (1.000 agent)

4.5 Demo koşusu ve animasyonlar

```
python experiments/run_demo.py
```

Küçük demo senaryosunu (`configs/demo.yaml`: 100 agent, 9 hücre, mobilite, yük) her yöntem için bir kez, bütün olayları loglayarak çalıştırır. Çıktı: `results/logs/demo/<yöntem>/{topology.json, events.jsonl, summary.json}`.

```
python viz/animate.py
```

İki video üretir (`results/figures/`):

- `agentroute_demo.mp4` / `.gif`: duyurular → bir isteğin hop hop yolculuğu → bir telefonun hücre değiştirmesi.
- `greedy_vs_agentroute.mp4` / `.gif`: aynı istek, aynı ağ, solda greedy sağda DQN. Sadece logda DQN'in gerçekten daha iyi olduğu bir istek varsa üretilir, sahnelenmez.

Sadece birini üretmek için `--only demo` ya da `--only compare` kullanılır. MP4 için `imageio-ffmpeg` paketinin içindeki ffmpeg kullanılır, ayrıca ffmpeg kurmak gerekmez.

4.6 Streamlit uygulaması

```
streamlit run viz/app_streamlit.py
```

Tarayıcıda `http://localhost:8501` açılır.

- **Sol menü:** agent sayısı, yöntemler, R, S-FIB özeti, hız, yük, çöken router oranı, süre ve seed seçilir. **Run simulation** ile çalıştırılır.
- **Sekmeler:**
  - **Results:** yöntem başına başarı, süre, hop ve mesaj sayısı tablosu, grafikler, başarısızlık nedenleri
  - **Network replay:** tek bir isteğin ağ üzerindeki yolu, o andaki yoğun router'lar
  - **Experiment figures:** `results/figures` içindeki bütün grafikler
  - **Web player:** koşuyu React oynatıcısına aktarır

AgentRoute seçenekleri ancak model eğitildikten sonra görünür. Replay için "Log every event" açık olmalıdır (1.000 agent'a kadar önerilir).

4.7 React + D3 log oynatıcısı

Önce Node.js LTS kurulmalı: nodejs.org → LTS → macOS `.pkg`.

Adım 1. Logları oynatıcıya aktar:

```
python viz/export_web_data.py
```

Adım 2. Bağımlılıkları kur (bir kere):

```
cd viz/web && npm install
```

Adım 3. Geliştirme sunucusunu başlat ve `http://localhost:5173` adresini aç:

```
cd viz/web && npm run dev
```

Kalıcı bir derleme için `npm run build` kullanılır. Çıktı `viz/web/dist/` klasörüne gider ve şöyle açılır:

```
python -m http.server 8000 -d viz/web/dist
```

Oynatıcıda:

- Üstte koşu seçilir, ya da `topology.json` + `events.jsonl` sürükle-bırak ile yüklenir.
- Ağ üzerinde paketler hop hop ilerler. Fare tekerleği zoom yapar, sürükleme kaydırır.
- **Zaman çizgisi:** oynat/duraklat, geri oynat (◀◀), başa dön, hız (0.01x–1x), kaydırıcı.
- **Request kartı:** bir istek seçilince hop başına gecikme şelalesi gösterilir. Bir pakete ya da olaya tıklamak o isteği takip eder.
- **Run summary:** başarı, süre, hop ve başarısızlık nedenleri.

5. Ayarlar (`configs/default.yaml`)

Anahtar	Varsayılan	Anlamı
`agents.n_agents`	100	agent sayısı
`agents.mobile_fraction` / `cloud_fraction`	0.5 / 0.1	mobil ve cloud agent oranı
`agents.class_zipf`	1.0	yetenek arzının Zipf üssü (0 = eşit dağılım)
`agents.speed_mps`	5	random-waypoint hızı
`agents.hysteresis`	0.15	handover histerezisi
`topology.agents_per_cell`	20	hücre sayısı agent sayısına göre büyür
`routers.hotspot_fraction`	0.15	arka plan trafiği olan router oranı
`sfib.radius`	3	duyuru yarıçapı R
`sfib.summary`, `sfib.k`	kmeans, 4	komşu başına özet
`forwarding.ttl`	16	paket TTL
`forwarding.tau`	null	null ise `results/tau.json`
`forwarding.header`	float32	intent header kodlaması
`sim.duration_s` / `query_rate` / `load`	30 / 10 / 1	süre, istek/s, yük çarpanı
`sim.failure_fraction`	0	çöken router oranı
`baselines.flood_ttl`	6	flooding TTL
`dqn.*`		eğitim hiperparametreleri ve reward ölçekleri

Kendi senaryon için sadece değiştirdiğin anahtarları içeren bir YAML yazıp `--config` ile verebilirsin (bkz. `configs/demo.yaml`).

Python içinden tek bir koşu:

```
from agentroute.config import load_config
from agentroute.sim.simulator import Simulator

cfg = load_config(overrides={"agents": {"n_agents": 1000}, "sfib": {"radius": 4}})
print(Simulator(cfg, "greedy", seed=1).run())
```

6. Log formatı

Simülasyon görselleştirmeden ayrıdır: simülatör olayları JSONL'e yazar, görselleştirmeler sadece bu dosyaları okur.

type	Alanlar	Ne zaman
`query`	pkt, agent, router, cls, text	istek oluştu
`hop`	pkt, from, to, method	paket bir linke çıktı
`deliver`	pkt, agent, hops	yetenekli agent'a ulaştı
`drop`	pkt, at, reason	başarısız (ttl, queue, link, wrong_agent, not_found, ...)
`handover`	agent, from_cell, to_cell	mobil agent hücre değiştirdi
`adv` / `withdraw`	from, to, n_vectors, card	S-FIB duyurusu / geri çekme
`move`	agent, x, y	mobil agent konumu (saniyede bir)
`load`	node, busy	hotspot router yoğunlaştı / rahatladı
`fail`	node	router çöktü

`topology.json` düğümleri (id, kind, x, y), linkleri (gecikme, kayıp) ve agent'ları (sınıf, kart metni, başlangıç konumu) içerir.

7. Klasör yapısı

```
agentroute/          çekirdek kod (dataset, embedding, S-FIB, forwarding, dqn/, sim/,
baselines/)
configs/             YAML ayarları
experiments/         deney script'leri (run_*.py, tune_tau.py, run_all.py)
viz/                 plots.py, animate.py, app_streamlit.py, export_web_data.py, web/
(React)
results/             tau.json, models/, raw/ (CSV), figures/, logs/
docs/                proposal, rapor, kılavuzlar, slaytlar, pdf/
tests/               pytest testleri
```

8. Sorun giderme

Belirti	Çözüm
Model indirilemiyor	İnternet bağlantısını kontrol et. Bir kere indikten sonra `~/.cache/huggingface` altından okunur.
`results/tau.json` bulunamadı	`python experiments/tune_tau.py` çalıştır.
`results/models/dqn.pt` bulunamadı	`python experiments/run_train.py` çalıştır, ya da sadece greedy/baseline yöntemlerini kullan.
MP4 üretilmedi, sadece GIF var	`pip install imageio-ffmpeg`
`npm: command not found`	Node.js LTS kur (nodejs.org).
Oynatıcıda "no exported runs"	`python viz/export_web_data.py` çalıştır.
10.000 agent deneyi yavaş	`--workers` değerini çekirdek sayısına çıkar ya da önce `--quick` dene.
Streamlit portu dolu	`streamlit run viz/app_streamlit.py --server.port 8502`